CHAPTER 14

Ruby from the Command Line

If you are using Ruby as a scripting language, you'll be starting it from the command line. In this chapter, we'll look at how to use Ruby as a command-line tool, and how to interact with your operating system environment. The two most common ways for a Ruby program to kick off from the command line are with the Ruby interpreter itself and with Rake, a utility that makes it easy to define a series of interrelated tasks. You also might want to create your own command-line programs, and Ruby can help with that as well.

Please note that some of the details of this chapter only apply to Unix-based systems like Linux, MacOS, and WSL.

Calling the Ruby Command

The most direct way to start the Ruby interpreter and run a Ruby program is by calling the `ruby` command from the command line. Regardless of the system in which Ruby is deployed, you have to start the Ruby interpreter somehow, and doing so gives us the opportunity to pass in command-line arguments both to Ruby itself and to the script being run.

A Ruby command line consists of three parts: options to the Ruby interpreter, optionally the name of a program to run, and optionally a set of arguments for that program, like this:

ruby ‹*options*› ‹–› ‹*programfile*› ‹*arguments*›*

The simplest Ruby command is just `ruby` followed by a filename:

```
$ ruby my_code.rb
```

This command will cause the Ruby interpreter to load the `my_code.rb` file, parse it, and then execute it.

If the file has a syntax error, Ruby will attempt to locate the error and suggest where the problem is:

Given this:

rubyworld/syntax_error.rb

```
class HasAnError
  def this_method_ends
    p "it sure does"
  end
```